

CLAUDE.md — project-er (이터널리턴 모작 포트폴리오)

프로젝트 개요

이터널리턴(Eternal Return)을 레퍼런스로 한 포트폴리오용 Unity 모작 프로젝트. 클라이언트 프로그래머 기술 어필을 목적으로 한다.

- **엔진:** Unity (URP)
- **언어:** C# (.NET Standard 2.1)
- **플랫폼:** PC (Windows)
- **라이선스:** MIT

구현 범위 (스코프)

핵심 구현 (반드시)

- 클릭투무브 이동 (NavMesh 기반)
- 캐릭터 1~2종 (기본 공격 + 스킬 포함)
- 기본 전투 (공격, 피격, 사망)
- 아이템 줍기 + 인벤토리 시스템
- 크래프팅 시스템 (재료 조합 → 아이템 제작)

선택 구현 (여유 시 추가)

- 몬스터 AI (순찰 → 어그로 → 추격)
- 미니맵
- 금지구역 시스템

제외 (스코프 아웃)

- 멀티플레이어 / 네트워킹
- 전체 맵 16구역 재현
- 캐릭터 40종 이상 구현

코딩 원칙

OOP 4대 원칙

- **캡슐화:** 필드는 `private` + `[SerializeField]`, 외부 접근은 프로퍼티로
- **상속보다 컴포지션** 우선 고려
- **다형성:** `if/else` 타입 분기 대신 인터페이스 + 오버라이딩 활용
- **추상화:** 구현 세부사항은 숨기고 인터페이스만 노출

SOLID 원칙

- **S (단일책임):** 클래스 하나 = 책임 하나. 역할이 늘어나면 클래스 분리
- **O (개방/폐쇄):** 기존 코드 수정 없이 확장 가능하도록 인터페이스 기반 설계
- **L (리스코프):** 자식 클래스는 부모를 완전히 대체 가능해야 함

- **I (인터페이스 분리)**: 인터페이스는 작게 분리, 불필요한 메서드 강제 구현 금지
- **D (의존성 역전)**: 구체 클래스 직접 의존 금지, 생성자 주입(DI) 방식 선호

코드 작성 규칙

네이밍 컨벤션

인터페이스	: I 접두사	→ IDamageable, IAttackable, ICraftable
추상 클래스	: Base 접미사	→ CharacterBase, ItemBase
컴포넌트 클래스	: 역할 명사	→ PlayerController, InventorySystem
ScriptableObject	: Data 접미사	→ ItemData, RecipeData, CharacterData
이벤트	: On 접두사	→ OnDeath, OnItemPickup, OnCraftComplete

필수 컨벤션

- **public** 필드 금지 → `[SerializeField]` **private** 사용
- `GetComponent<T>()` 대신 `TryGetComponent<T>()` 사용
- 런타임에서 `Find()`, `FindObjectOfType()` 호출 금지 → 직접 참조 또는 DI
- 태그 비교는 `CompareTag()` 사용 (string 할당 방지)
- 이벤트 구독/해제는 `OnEnable` / `OnDisable` 쌍으로
- 매직 넘버 금지 → `const` 또는 `[SerializeField]` 상수로 선언

하지 말 것

- **God Class** 금지: GameManager 하나에 모든 로직 몰아넣기 금지
- **static** 남용 금지: 전역 상태가 필요하면 ScriptableObject 이벤트 채널 사용
- **Update()** 내 GC 유발 코드 금지: LINQ, string 연결, `new` 컬렉션 생성 금지
- **컴포넌트 캐싱** 누락 금지: 참조는 `Awake()/Start()`에서 캐싱

시스템별 설계 가이드

이동 시스템 (Click-to-Move)

NavMesh 기반 클릭투무브

- 마우스 우클릭 → Raycast → `NavMeshAgent.SetDestination()`
- NavMeshAgent 설정: `stoppingDistance`, `speed`는 `CharacterData(SO)`에서 읽음
- 스킬 대기/순간이동: NavMeshAgent 일시 정지 후 직접 위치 이동

전투 시스템

인터페이스 설계:

- IDamageable : `TakeDamage(float amount)`
- IAttackable : `Attack(IDamageable target)`
- ISkillUser : `UseSkill(int skillIndex)`

```
CharacterBase (MonoBehaviour + IDamageable):
├─ PlayerController : CharacterBase, ISkillUser
└─ MonsterController : CharacterBase, IAttackable
```

스탯: CharacterData(ScriptableObject)에서 읽어옴

상태 관리: enum CharacterState + StateMachine 구조 사용 (bool 플래그 남발 금지)

인벤토리 & 아이템

데이터 레이어:

- ItemData (ScriptableObject): ID, 이름, 아이콘, 타입, 스탯 보너스
- ItemDatabase (ScriptableObject): List<ItemData> + Dictionary 조회용 캐시

런타임 레이어:

- InventorySystem: 슬롯 관리, 아이템 추가/제거/이동
- InventorySlot: 아이템 참조 + 수량

이벤트: ScriptableObject 이벤트 채널 방식 권장

크래프팅 시스템

RecipeData (ScriptableObject):

- List<ItemIngredient> ingredients (ItemData + 수량)
- ItemData resultItem
- int resultAmount

CraftingSystem:

- RecipeDatabase에서 레시피 조회
- 인벤토리에 재료 충분한지 검증 후 제작
- 조회 성능: Dictionary<string, RecipeData> 로 캐싱

Claude에게 지시사항

코드 생성 시

2. 새 클래스 작성 전 인터페이스 설계 먼저 제안
3. MonoBehaviour 생성 시 생명주기 스텝(Awake, OnEnable, OnDisable, OnDestroy) 포함
4. GC 할당 가능성 있는 코드엔 // ⚠ GC 주의 코멘트 추가
5. Unity 버전 종속 API 사용 시 버전 명시

리팩터링 제안 시

- [SerializeField] 필드명 임의 변경 금지 (Inspector 직렬화 데이터 손실)
- 동일 패턴이 3회 이상 반복되면 반드시 추상화 제안
- 타입 분기(if type == ...)가 보이면 다형성 리팩터링 제안

응답 언어

- 코드 내 주석: 한국어
- 설명 텍스트: 한국어
- 변수/클래스/메서드명: 영어

프로젝트 구조 (참고)

```
Assets/
├── Scripts/
│   ├── Character/
│   │   ├── CharacterBase.cs
│   │   ├── Player/
│   │   └── Monster/
│   ├── Combat/
│   │   ├── IDamageable.cs
│   │   └── IAttackable.cs
│   ├── Inventory/
│   │   ├── InventorySystem.cs
│   │   └── InventorySlot.cs
│   ├── Crafting/
│   │   └── CraftingSystem.cs
│   ├── Data/          ← ScriptableObject 데이터 클래스
│   └── UI/
├── ScriptableObjects/
│   ├── Items/
│   ├── Recipes/
│   └── Characters/
└── Prefabs/
```